IARTI report

Scheduling algorithm report

Bernardo Paiva do Novo Granja Cardoso [1], Francisco Barradas Lemos Lousada [2] , Rui Manuel Pinto Santiago[3] and Tiago de Barros Simões [4]

[1] ISEP; 1231090@isep.ipp.pt
[2] ISEP; 1231092@isep.ipp.pt
[3] ISEP; 1221402@isep.ipp.pt
[4] ISEP; 1231108@isep.ipp.pt

1. Introduction

This work presents a dedicated Planning & Scheduling backend module that computes loading and unloading plans for vessels using a REST-based API. The module retrieves all required operational data from existing backend services and performs stateless, on-demand scheduling without persisting results. An optimal algorithm was implemented to minimize vessel departure delays, followed by a greedy algorithm to offer a faster option capable of realistically handling big amounts of data. Finally, the system was extended to support multi-crane scheduling when single-crane solutions are insufficient. This document aims to describe and **analyse the complexity of this solution.**

2. Materials and Methods

The development of the Planning & Scheduling module relied on several components and tools, the main being the Prolog programming language, which was used throughout the entire solution to implement all the algorithm logic and server architecture. The server architecture follows a REST-Based framework by handling data as a stateless API, without persisting any information. All input and output of the server is handled through JSON data structures.

Besides internal logistics, the server also uses our own Backend API of the port managing system to feed its algorithms with relevant data such as:

1. The arrival date of vessels
2. The departure date of vessels
3. The operational window of cranes and relevant staff
4. The contents of a vessel visit's cargo manifest
5. The quantity of cranes operating at a certain dock
6. The speed of those cranes

The format of this data will become relevant in the coming chapters, when a scheduling request is made for a certain day and a certain dock. Below is a sample of what could be returned by the Backend API:

```
{
  "craneWorkloads": [
    {
      "crane": "string",
      "speed": 0,
      "operatingWindow": {
        "shifts": [
          {
```

```
            "day": 0,
            "startTime": "string",
            "endTime": "string"
          }
        ]
      }
    }
  ],
  "vesselTaskFacts": [
    {
      "vessel": {
        "name": "string",
        (…)
      },
      "eta": 0,
      "etd": 0,
      "loadingCount": 0,
      "unloadingCount": 0
    }
  ],
  "comment": "string"
}
```


3. Results

This section will skip over how requests are processed using the HTTP protocol and other superficial networking tasks, as it is outside the scope of this analysis; as well as other architectural systems such as "how the algorithm is selected" or "how is the relevant data fetched from the backend"

3.1. Deviations from the provided algorithms

Despite using the algorithm templates provided by the course, we had to slightly alter them to better fit our needs (especially after introducing multi-crane scheduling); the most major alteration was one to the `vessel/5` predicate signature, we changed it to be a `vessel/6`, where the first three parameters (like in the template) represent the vessel name, the estimated time of arrival, and the estimated time of departure, but the last three parameters were changed to be the number of containers to load (as opposed to the speed of loading all containers) and the number of containers to unload (as opposed to the speed of unloading all containers), the last parameter which was not included in the original template, is a list with the cranes that did the loading/unloading operations on this specific vessel.

In this system we task the algorithm itself with calculating the speed of loading/unloading using the crane list, the number of containers to load/unload and the `crane/2` predicate, which relates the name of a crane to its speed. So, each time an algorithm tests a permutation of cranes on a vessel, it is required to calculate the unload & load times on the fly using the following formula:

$$LoadingTime = \frac{loadCount}{\sum craneSpeeds} \quad UnloadingTime = \frac{unloadCount}{\sum craneSpeeds}$$

3.2. Optimal scheduling algorithm

We'll spare the reader the details of this implementation as it is quite literally the exact same logic as the one provided to us in the theoretical classes. The dominant cost of this algorithm comes from trying all permutations of vessels and evaluating each sequence.

The algorithm begins at `obtain_seq_shortest_delay/2` but to facilitate the analysis we will begin with the predicates that don't depend on others, that will be `sequence_temporization/3` and `sum_delays/2`, which only use our custom speed calculation predicates: `calculate_load_unload_time/5` and `get_crane_sum/2`; se let us analyse those first:

```
get_crane_sum([], 0).
get_crane_sum([crane(_, Speed) | RestCranes], Sum):-
    get_crane_sum(RestCranes, Sum1),
    Sum is (Sum1 + Speed).
```

The `get_crane_sum/2` algorithm takes in a list of cranes and a result by reference, it then iterates over the list of cranes provided and sums their speeds onto the result, this one should be a fairly clear example of an O(n) function, being n the number of cranes.

```
calculate_load_unload_time(LoadCount, UnloadCount, CraneList, LoadTime, UnloadTime):-
    get_crane_sum(CraneList, Sum),
    ( (LoadCount > 0, Sum > 0) -> LoadTime is (LoadCount / Sum) ; LoadTime = 0 ),
    ( (UnloadCount > 0, Sum > 0) -> UnloadTime is (UnloadCount / Sum) ; UnloadTime = 0 ).
```

The `calculate_load_unload_time/5` merely applies the formulas as described on the previous chapter. It does call `get_crane_sum/2` which makes it O(n) by transition.

```
sum_delays([],0).
sum_delays([(V,_,TEndLoad)|LV],S):-
   vessel(V,_,TDep,_,_,_),TPossibleDep is TEndLoad+1,
   ( (TPossibleDep>TDep,!,SV is TPossibleDep-TDep);SV is 0),
   sum_delays(LV,SLV),
   S is SV+SLV.
```

Now onto `sum_delays/2`, this one is also fairly trivial to analyse. It calls itself once per vessel but besides that has only constant time work, making it also O(n) being n the number of vessels.

```
compare_shortest_delay(SeqTriplets,S):-
      shortest_delay(_,SLower),
      ((S<SLower,!,retract(shortest_delay(_,_)),asserta(shortest_delay(SeqTriplets,S)));true).
```

This is a helper predicate that only performs one comparison and retract/asserts, in Prolog knowledge base operations are mostly constant time, so it will end up at O(1).

```
sequence_temporization(LV,SeqTriplets):-
 sequence_temporization1(0,LV,SeqTriplets).

sequence_temporization1(EndPrevSeq,[V|LV],[(V,TInUnload,TEndLoad)|SeqTriplets]):-
    vessel(V,TIn,_,TUnloadC,TLoadC, Cranes),
    calculate_load_unload_time(TUnloadC,TLoadC, Cranes,TUnload, TLoad),

    ( (TIn> EndPrevSeq,!, TInUnload is TIn); TInUnload is EndPrevSeq+1),
```

```
    TEndLoad is TInUnload + TUnload+TLoad -1,
    sequence_temporization1(TEndLoad,LV,SeqTriplets).

sequence_temporization1(_,[],[]).
```

This is one of the main predicates for the algorithm, it tries to line up the vessels schedule, so it calls itself once per vessel, it also calls `calculate_load_unload_time/5` every iteration, making it $O(n \times c)$, where n is the number of vessels and c is the number of cranes being tested (worst case all of them).

```
    obtain_seq_shortest_delay(SeqBetterTriplets, SShortestDelay):-
        (obtain_seq_shortest_delay1;true),retract(shortest_delay(SeqBetterTriplets,  SShortestDe-
lay)),!.

    obtain_seq_shortest_delay1:-
        asserta(shortest_delay(_,100000)),
        findall(V,vessel(V,_,_,_,_,_),LV),
        permutation(LV,SeqV),
        sequence_temporization(SeqV,SeqTriplets),
        sum_delays(SeqTriplets,S),
        compare_shortest_delay(SeqTriplets,S),
        fail.
```

At last, there is the biggest bottleneck of the algorithm, the predicate that attempts all permutations and finds the best one. It starts with three preparatory instructions and then enumerates every permutation of the vessel list, for each permutation it calls `sequence_temporization/3` and `compare_shortest_delay/2`. Let n be the number of vessels and c be the number of cranes processed for a single vessel; the cost of `findall/3` is O(n), `permutation/2` generates $n!$ Sequences. For a single permutation the algorithm runs:

1. `sequence_temporization/3`: $O(n \times c)$
2. `sum_delays/2`: $O(n)$
3. `compare_shortest_delay/2`: $O(1)$

Hence the cost of a single permutation is of $O(n \times c + n) = O(n \times c)$, multiplying by the number of permutations gives the total time complexity of: $O(n! \times (n \times c))$

3.3. Greedy scheduling algorithm

We also implemented a greedy scheduling algorithm that prioritizes the earliest due date, meaning the earliest departure.

This means that, before calculating, the algorithm collects and sorts all the vessels, in ascending order, by their departure hour, that being its heuristic. After that, the algorithm begins attributing the in and out times for each vessel, in the sorted order: if the vessel hasn't arrived yet, it starts when it arrives; otherwise, it starts right away. The vessel then finishes its operation, and we calculate the arrival of the next vessel, and whether it had to wait or not. After all vessel times are calculated, the algorithm also calculates the total delay and returns it.

We assumed that the average port, in a week, would see about 20 vessel berths per dock, after conducting superficial research. According to that, we measured this algorithm's performance while increasing the number of vessels, and plotted the results:

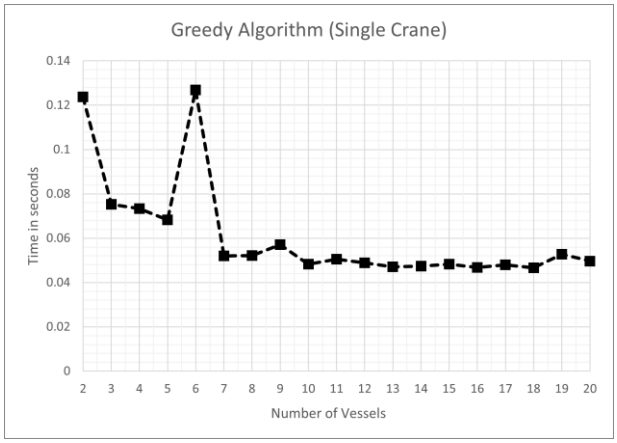

Figure 1 – Greedy algorithm processing time variation with increasing vessels (20).

As we can see, the processing time is so little that our captured data appears to fluctuate due to imprecisions in the measuring method used and doesn't represent the actual processing time accurately. To overcome this, we tried increasing the number of vessel facts tested to 100, and we achieved the following:

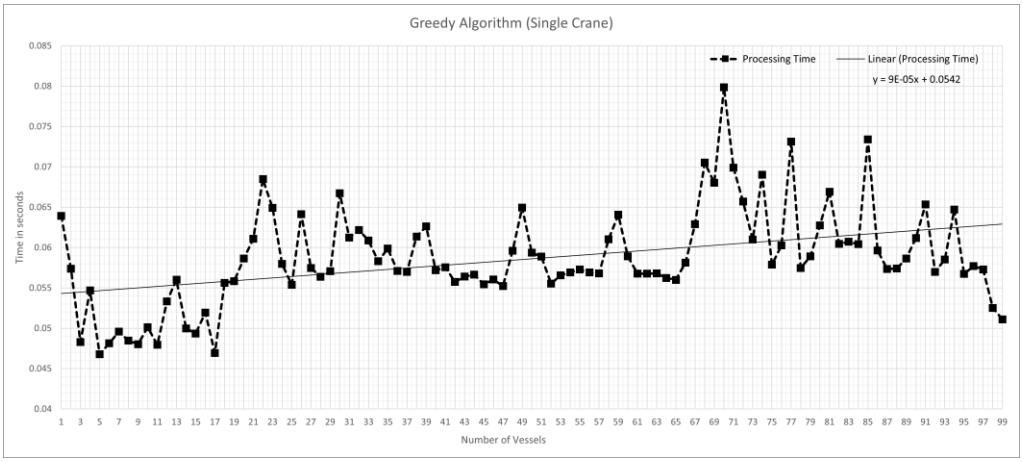

Figure 2 – Greedy algorithm processing time variation with increasing vessels (100).

The data, while still noisy, appears to present a slight trend of about $9 \times 10^{-5}N$, leading us to assume that this algorithm is O(N), but more on that later.

We also compared this algorithm to the Optimal algorithm, which is, as the name implies, optimal in terms of results, but not so much in terms of processing time:

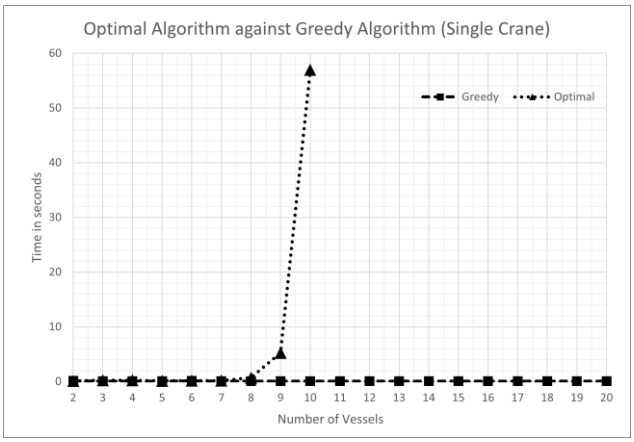

Figure 3 – Greedy algorithm processing time compared to Optimal algorithm.

It seems fair to say that the implementation of the greedy algorithm, while not perfect, seems to provide a sufficiently good solution to the proposed problem. While the Optimal algorithm fails to calculate any solution past 10 vessels due to hardware and software limitations, the Greedy algorithm continues to calculate valid schedules with very steady computation times.

Even when compared in terms of delays, the Greedy algorithm keeps up, with the difference only increasing quadratically, which seems very reasonable for a discrepancy of X in terms of factors of N:

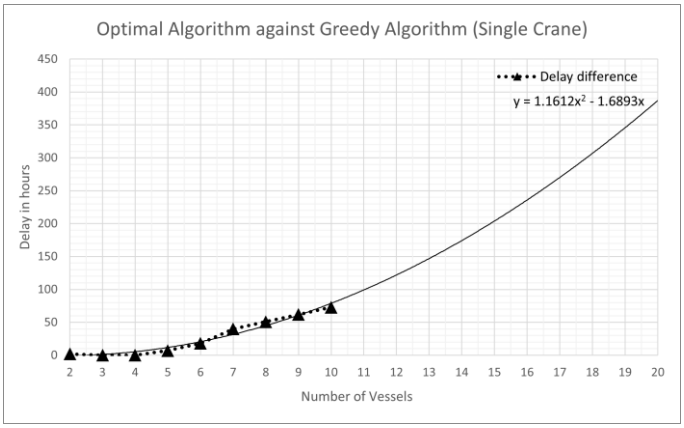

Figure 4 – Delay difference between Optimal and Greedy algorithms compared to vessels.

Now, let's calculate the time complexity of the Greedy algorithm. Our algorithm begins by collecting all known vessel facts with a `findall/3`, that has a complexity of $O(N)$. Then, those vessel facts are sorted using SWI Prolog's `sort/2`, that has a time complexity of $O(N\ log\ N)$. The algorithm then: looks up a `vessel/6` fact, $O(1)$; iterates through the list of cranes, $O(K)$; does some arithmetic logic, $O(1)$. After the algorithm is done, it also calculates the total delays, by iterating through all the vessels, $O(N)$.

This brings us to a complexity of $O(N) + O(N\ log\ N) + O(NK) + O(N)$, we can then reduce this to $O(N\ log\ N + N \times K)$, and, since this all happens on a single dock, meaning the number of cranes K is constant and equal to all vessels, we can further reduce this to $O(N\ log\ N)$, meaning we're bottlenecked by the `sort/2` method used.

So, recalling our previous time complexity assumption, it's not farfetched, and, realistically, the number of vessels is always going to be low enough that N and $N\ log\ N$ aren't that far, since for $N = 100$, $N\ log\ N = 664$, making that a ratio of 6.64. Our algorithm is never going to see that scale of data, and quite frankly, even the N constant is negligible, since the previously captured trend factor of 9×10^{-5} implies that each vessel takes approximately 0.09 milliseconds, or 90 microseconds. That's about as fast as page fault that makes the computer read from disk paged memory, or about 10^{158} times faster than the "Optimal" algorithm.

3.4. Multi-crane scheduling

After the implementation of both algorithms, we implemented multi-crane scheduling, that technically is not a separated algorithm, is the adaptation of both algorithms to use multiple cranes. With that in mind, the complexity of both stills the same since we analyzed it with the multi-crane already implemented. The following code is the main entry point with resource setup:

```
    schedule_daily_operations(TargetDate, DaysAhead, DockCode, Algorithm, Sched-
uleResult, Metrics) :-
        get_vvns_on_day(TargetDate, DaysAhead, DockCode, JsonData),
```

```
    date_weekday(TargetDate, Weekday),
    assertz(current_schedule_day(Weekday)),
    schedule_daily_operations1(JsonData, Algorithm, ScheduleResult, Metrics),
    retractall(current_schedule_day(_)).
```

In this block is where we start by fetching the vessel visit notification from the data source, then we determine what day of the week it is and store it as a temporary global artifact in Prolog's database (so other predicates can access it), after that we run the actual scheduling logic were the validations about what scheduling to choose are. In the end we clean up the temporary weekday fact, the use of `assertz` and `retractall` ensures the weekday information is available during scheduling but doesn't pollute the database afterwards.

```
schedule_daily_operations1(JsonData, _, 'Missing resource (qualified staff or STS cranes on dock)', Metrics) :-
    { error: _ } :< JsonData, !,
    Metrics = #{algorithm: none, totalDelay: 0, computationTime: 0, vesselCount: 0}.
```

When the schedule_daily_operations1 is called, in case of an error it catches that condition before attempting scheduling. If the JSON data contains an error field (indicating missing staff or cranes), it immediately returns a failure message with zeroed metrics. The ! (cut) prevents backtracking, making this a definitive error exit.

```
schedule_daily_operations1(JsonData, Algorithm, ScheduleResult, Metrics) :-
    extract_scheduling_data(JsonData, VesselFacts, CraneFacts),
    retractall(vessel(_,_,_,_,_,_)),
    retractall(crane(_,_)),
    flatten(VesselFacts, FlatVesselFacts),
    flatten(CraneFacts, FlatCraneFacts),
    maplist(assertz, FlatCraneFacts),
```

In case of succeeding the real predicate logic begins, the algorithm starts by extracting the data received from the JSON using the method `extract_scheduling_data` It's a simple method just to extract the data received from the JSON, it uses a predicate from our vvn_mapper named `json_to_vvn_fact`, that just assigns the data to the respective variables. After that it clears any existing vessel and crane facts from the database to start fresh, then it flattens potentially nested lists into simple flat lists of facts and adds all crane facts to the Prolog database using `assertz`.

```
format(user_error, '~n=== PHASE 1: Single-crane scheduling ===~n', []),
assert_single_crane(FlatVesselFacts),
run_algorithm(Algorithm, SingleRaw, SingleDelay, SingleTime),
format(user_error, 'Single-crane delay: ~w~n', [SingleDelay]),
capture_assignments(SingleRaw, SingleResult),
```

After that phase 1 can start, it quickly generates a baseline solution by assigning exactly one crane per vessel (the fastest available). This is a greedy approach that's computationally cheap but may not be optimal. It also measures the total delay and computation time. In the case of the delay being greater than 0, phase 2 (multi-crane scheduling) can start, if its 0 then it ends and sets the result produced by the single crane scheduling.

```
( SingleDelay > 0 ->
        format(user_error, '~n=== PHASE 2: Multi-crane permutations ===~n',
        []),
```

```
            retractall(vessel(_,_,_,_,_,_)),
            get_time(T1),
            find_best_assignment(FlatVesselFacts, Algorithm, MultiRaw, MultiDe-
        lay),
            get_time(T2),
            MultiTime is T2 - T1,
            capture_assignments(MultiRaw, MultiResult),
```

This code only runs if there are delays in phase 1, it explores different crane assignment permutations to find a better solution. It's more computationally expensive, so it's skipped when unnecessary (zero delays).

```
    ( MultiDelay < SingleDelay ->
                format(user_error, 'Multi-crane BETTER! Improvement: ~w~n', [Sin-
            gleDelay - MultiDelay]),
                ScheduleResult = MultiResult,
                TotalDelay = MultiDelay,
                ComputationTime = MultiTime,
                Strategy = 'multi-crane'
            ;
                ScheduleResult = SingleResult, TotalDelay = SingleDelay,
                ComputationTime = SingleTime, Strategy = 'single-crane'
            )
    ;    format(user_error, 'Single-crane has NO delays! Using single-crane~n',
[]),
            ScheduleResult = SingleResult, TotalDelay = SingleDelay,
            ComputationTime = SingleTime, Strategy = 'single-crane'
     ),
```

Basically, the purpose of this is to make an efficient strategy selection, if phase 1 had no delays we skip phase 2, if the phase 1 had delays and phase 2 found a better solution we choose the phase 2 solution and finally if phase 1 had delays but phase 2 didn't improve we keep the single crane solution. This ensures the system always returns the best solution while avoiding unnecessary computation.

Previously some predicates were called, these are the most important ones:

```
    assert_single_crane([]).
    assert_single_crane([vessel(Name, Arrival, Departure, Unload, Load, _)|Rest])
:-
        findall(crane(CraneName, Speed), crane(CraneName, Speed), AllCranes),
        AllCranes = [crane(FastestName, FastestSpeed)|_],
        format(user_error, 'Vessel ~w assigned crane: ~w (speed: ~w)~n', [Name,
    FastestName, FastestSpeed]),
        assertz(vessel(Name, Arrival, Departure, Unload, Load, [crane(FastestName,
    FastestSpeed)])),
        assert_single_crane(Rest).
```

Recursively processes each vessel and assigns it the fastest available crane. This implements the greedy single-crane strategy. The code assumes cranes are pre-sorted by speed (fastest first), so it just takes the first crane from the list for each vessel.

Other important methods are `find_best_assignment`, `test_assignments` and `test_helper`. We will start with the `find_best_assignment`:

```
    find_best_assignment(VesselFacts, Algorithm, BestResult, BestDelay) :-
```

```
        findall(crane(CraneName, Speed), crane(CraneName, Speed), AllCranes),
        length(AllCranes, NumCranes),
        length(VesselFacts, NumVessels),
        format(user_error, 'Trying ~w cranes for ~w vessels~n', [NumCranes, NumVes-
    sels]),
        gen_assignments(NumVessels, NumCranes, Assignments),
        length(Assignments, TotalAssignments),
        format(user_error, 'Total assignments: ~w~n', [TotalAssignments]),
        test_assignments(Assignments, VesselFacts, AllCranes, Algorithm, BestRe-
    sult, BestDelay).
```

This is the main coordinator for Phase 2 optimization. It collects all available cranes from the database, then counts the resources (how many vessels need scheduling and how many cranes are available), after that generates all possible assignments calling `gen_assignments` (simple method that generates combinations) to create every possible way to distribute cranes to vessels. Then it tests all permutations with `test_assignments` evaluating each one. The assignment with the lowest delay or, in case of equal delays, the one with fewer cranes is chosen.

```
    gen_assignments(0, _, [[]]) :- !. gen_assignments(NumVessels, MaxCranes, As-
signments) :-
    NumVessels > 0,
    NumVessels1 is NumVessels - 1,
    gen_assignments(NumVessels1, MaxCranes, RestAssignments),
    findall([CraneCount|RestAssignment], (between(1, MaxCranes, CraneCount), mem-
ber(RestAssignment, RestAssignments)), Assignments).
```

So, analysing the complexity, for V vessels and C cranes, generates C^V assignments (exponential growth). The predicate generates assignments for N-1 vessels and for each existing assignment, add every possible crane count (1 to MaxCranes), this creates a Cartesian product.

Following to `test_asignments`:

```
    test_assignments([FirstAssignment|RestAssignments], VesselFacts, AllCranes,
Algorithm, BestResult, BestDelay) :-
        retractall(vessel(_,_,_,_,_,_)),
        apply_assignment(VesselFacts, FirstAssignment, AllCranes),
        run_algorithm(Algorithm, FirstResult, FirstDelay, _),
        sum_list(FirstAssignment, FirstTotal),
        format(user_error, '~w (total: ~w) -> Delay: ~w~n', [FirstAssignment,
FirstTotal, FirstDelay]),
        test_helper(RestAssignments, VesselFacts, AllCranes, Algorithm, FirstRe-
sult, FirstDelay, FirstAssignment,
                    BestResult, BestDelay, BestAssignment),
        sum_list(BestAssignment, BestTotal),
        format(user_error, '~n=== SELECTED: ~w (total: ~w) delay: ~w ===~n', [Bes-
tAssignment, BestTotal, BestDelay]),
        retractall(vessel(_,_,_,_,_,_)),
        apply_assignment(VesselFacts, BestAssignment, AllCranes),
        run_algorithm(Algorithm, BestResult, BestDelay, _).
```

It starts the evaluation process and handles the result. First, it tests the first assignment by clearing every vessel, then applies the first crane distribution and run scheduling

algorithm. Then passes the rest of the assignments and the current best to the `test_helper` for it to return the real best assignment. In the end, clears vessels again, apply the best found and re-run the algorithm one more time with the winning configuration. The purpose of re-running at the end is because `test_helper` only tracks which assignment is best but doesn't preserve the final vessel state in the database. The last three lines reconstruct that state.

Finally, the `test_helper` predicate:

```
test_helper([], _, _, _, CurrentBest, CurrentDelay, CurrentAssignment, CurrentBest, CurrentDelay, CurrentAssignment) :- !.
test_helper([Assignment|RestAssignments], VesselFacts, AllCranes, Algorithm, CurrentBest, CurrentDelay, CurrentAssignment, BestResult, BestDelay, BestAssignment) :-
    retractall(vessel(_,_,_,_,_,_)),
    apply_assignment(VesselFacts, Assignment, AllCranes),
    run_algorithm(Algorithm, Result, Delay, _),
    sum_list(Assignment, TotalCranes),
    sum_list(CurrentAssignment, CurrentTotal),
    format(user_error, '~w (total: ~w) -> Delay: ~w~n', [Assignment, TotalCranes, Delay]),
    ( Delay < CurrentDelay ->
        format(user_error, '  -> BETTER: Lower delay~n', []),
        NewBest = Result, NewDelay = Delay, NewAssignment = Assignment
    ; Delay =:= CurrentDelay, TotalCranes < CurrentTotal ->
        format(user_error, '   -> BETTER: Fewer cranes (~w < ~w)~n', [TotalCranes, CurrentTotal]),
        NewBest = Result, NewDelay = Delay, NewAssignment = Assignment
    ;   NewBest = CurrentBest, NewDelay = CurrentDelay, NewAssignment = CurrentAssignment
    ),
    test_helper(RestAssignments, VesselFacts, AllCranes, Algorithm, NewBest, NewDelay, NewAssignment,
                BestResult, BestDelay, BestAssignment).
```

Recursively tests each remaining assignment and maintains the running winner. It starts by applying the next assignment (clear vessels and distribute cranes according to the assignment pattern), then it calculates the delay for this configuration running the scheduling algorithm. After it compares the current best, first compares the lower delay and in case of being equal the one with fewer cranes wins ensuring resource efficiency. Then it continues with remaining assignments passing the “possibly new” best.

4. Conclusions

In practical scheduling environments, solution time is often a hard constraint. This means that an algorithm capable of producing a good-quality schedule quickly is typically more valuable than one that guarantees optimality but becomes computationally infeasible as the problem grows.

We feel that the Greedy algorithm is better qualified for the task of calculating a valid schedule in a reasonable amount of time, while in a theoretical, infinitely long amount of time, the Optimal algorithm would almost always be better.